

Offline states and sync conflicts · Dark patterns: naming and judging

What is this sheet about?

This sheet asks two questions of your own prototype: does it stay usable when the network disappears, and does it stay honest when the metrics tempt you? You triage which functions must survive offline, design the conflict case for your most critical data type, and file your own screens on the scale from legitimate to forbidden. Your partner's job is to attack both lists.

Organisation: Work in teams of three, four only if the numbers do not divide. The partner test in assignment 2 and the crossover in assignment 4 need exactly one counterpart, so you pair up inside the team: a team of four splits into two pairs, a team of three swaps in a circle (A tests B's, B tests C's, C tests A's). On this sheet, the letters A to D stand for the team members; settle who is who before the first assignment. The design work is individual (your own seminar project); your counterpart tests it and pushes back.

What you take with you: At the end of the session, **every person** holds their own offline one-pager (assignments 1 and 2), triple filing (assignment 3) and crossover finding (assignment 4). The sheet has no upload of its own; all three are material for Part C (upload Fri 18 Dec) and the seminar paper (due Fri 15 Jan 2027).

First assignment (30 minutes): Topics 9 and 10 put two questions to your own prototype: does it stay *usable* when the network disappears, and does it stay *honest* when the metrics tempt you?

Start with the airplane-mode tests. The bridge task from Topic 8 is voluntary and goes on the Padlet, so this sheet does not wait for it: if you did the test, show the first break you found – what stopped working, and what did the app say about it? If you did not, switch your phone to airplane mode now, open an app you use daily and find the first break; two minutes are enough. Without a device, take one of the two prepared cases from the Topic 9 slides. Sort the break into one of the three columns from Topic 9: silently broken, honestly broken, kept working. Two minutes per person. Then run the triage for your own prototype: name the three functions that must survive offline, and the one function that honestly does not have to. Four lines, one per function. With the fourth line you name what you give up offline. Your partner's job: try to argue one of your three functions *out* of the list, or the fourth one *in*. Only a list that survives this attack goes into the documentation.

Second assignment (60 minutes): Design the conflict case for your most critical data type. The situation: two devices, two divergent states. Your user edited offline on one device while the other device changed the same data. Design three screens: *detection* (how does the user learn there is a conflict?), *decision* (what are they asked, in which words?), *confirmation* (what tells them it is resolved, and what happened to the losing version?). Sketch or tool, fidelity free. Check the screens against the line from Topic 9: the user must see that a conflict happened, what is at stake and what happened to their work; they must never see vector clocks, replica IDs or merge internals.

Then choose your resolution strategy from Topic 9 (last-writer-wins, merge, ask the user, or block offline writes) and name its failure mode: the concrete situation, for your data type, in which this strategy quietly does the wrong thing. A generic failure mode ("data might get lost") fails the sheet.

Test the three screens with your partner: the partner plays the user who made the offline edit and walks through the screens aloud. Note where they hesitate, and what they believe happened to their data afterwards. If what they believe differs from what your design did, change the screen.

Third assignment (45 minutes): Switch from the offline test to honesty. Take your borderline case: the pattern in one of your own apps where you are honestly unsure whether it is persuasion or manipulation. It comes from the borderline-case bridge (end of Topic 8 this term), which is voluntary and goes on the Padlet. If you have none, take one of the two 2023 candidates from the Topic 10 slides: Bakkt's checklist at "0 % Completed" or Vivid's pre-selected free Prime month. File it individually on three levels with the tools from Topic 10 and keep the levels apart; mixing them is the failure mode:

1. **Taxonomy:** the Brignull type (deceptive.design) plus the Gray et al. ontology level (strategy / meso / implementation).
2. **Legal anchor:** which norm plausibly applies (DSA Art. 25, UCPD, GDPR, or Data Act Art. 6), or none, and why. Name the norm and give the reason it applies.
3. **Your verdict:** persuasion or manipulation, three sentences of justification along the four criteria (intent, harm, asymmetry, deception).

AI is allowed as a tool for the legal lookups, but every citation is checked against the original before it enters your filing.

Fourth assignment (30 minutes): The crossover: your conflict dialog from the second assignment is itself a pattern. It has a default, and defaults steer. Exchange your conflict-case designs inside the team – in a team of three, in the same circle as in assignment 2 – and file your counterpart's *decision* screen with the same three-level discipline: Whose interest does the default serve, the user's data or the app's simplicity? Is the losing version visible, or quietly discarded? Would you call the screen persuasive, manipulative, or neutral, and by which of the four criteria? One short paragraph of findings per person. If your counterpart found a nudge in your screen, decide: keep it and defend it in one sentence, or change the screen.

Keep the offline one-pager (triage and conflict-case design with the named failure mode), the triple filing and the crossover finding. They go into Part C and the seminar paper wherever your project takes them up.